Task Management Web Application

Database Schema

The application uses a database to store information, which is managed using SQLAlchemy. The database has two main tables: Users and Tasks.

1. Users Table

This table stores information about user accounts.

It contains the following fields:

- ID – a unique number for each user (primary key)
- Username – must be unique and cannot be empty
- Email address – must be unique and cannot be empty
- Password – stored securely using a password hashing method and cannot be empty

2. Tasks Table

This table stores the tasks created by users.

It includes the following fields:

- ID – a unique number for each task (primary key)
- user_id – links the task to a specific user (foreign key)
- Title – required field
- Description – additional details about the task
- Due date – required field
- Status – required field
- Priority – required field

Relationship Between Users and Tasks

The user_id connects tasks to the user who created them.

This creates a relationship where: One User → Can Have Many Tasks

In simple terms, each user can create multiple tasks, but every task belongs to only one user.

Key Design Decisions

Flask-Login: Flask-Login was used to manage user authentication and sessions, allowing users to securely log in and access only their own tasks. It also protects certain routes so that only authenticated users can view or modify task data.

Werkzeug – Security: This library was used to store the password securely via password_hash() method.

SQLAlchemy: SQLAlchemy was used to interact with the database using Python classes instead of raw SQL queries. This improves readability and maintainability.

Modular Code Structure: The application separates functionality into different components such as routes, templates, and models. This structure makes the code easier to maintain and extend.

WTForms: WTForms was used to handle form input and validation for features such as registration, login, and task creation. It helps ensure users enter valid data and improves security by providing built-in validation and CSRF protection.

Form Handling: HTML forms are used to collect user input for creating and editing tasks. The server processes these requests using POST methods.

Flash Messaging: Flash messages were implemented to provide immediate feedback to users after actions such when the username or email address is not unique for registering, incorrect details for login, tasks creation and when due date is selected in the past.

Challenges Faced and Solutions

During development, one challenge was that flash messages were not displayed, even though flash() was used in the Flask routes. The issue was resolved by adding the get_flashed_messages() block in the HTML templates so the messages could be retrieved and displayed. Another challenge occurred during testing when required fields such as due dates and priority were missing from the test data, causing validation errors. This was fixed by including all required fields and ensuring date values were correctly handled as Python date objects when interacting with the database. Another challenge involved handling date values in SQLAlchemy. SQLite requires date fields to be stored as Python date objects rather than strings. This required converting string dates into Python date objects when creating database entries directly during tests. I also discovered that priority and status filtering was returning tasks for all users due to overwritten queries. I then resolved it by chaining SQLAlchemy filter_by() calls instead of resetting the query.

Advanced Features Implemented

Several advanced features were implemented in the application.

User Authentication: The system allows users to register and log in to their own accounts. Authentication ensures that each user can only access their own tasks.

Personalised Homepage: The home page displays a generic message for guests and a personalised welcome message including the user's username when logged in.

Task Count Display The dashboard dynamically calculates and displays the total number of tasks created by the logged-in user.

Empty Dashboard Message: When a user has no tasks, the dashboard displays a message saying *"You have 0 tasks. Create your first task!"* to guide the user to start adding tasks.

Task Editing and Deletion: Users can update task details or remove tasks when they are no longer needed.

Task Filtering: Users can filter tasks by status and priority, with a clear filter button (Show All Tasks) to quickly reset the filters.

Task Sorting: Tasks can be sorted by due date, title, or status to help users organise and prioritise their work.

Task Search: Users can search for tasks using keywords in the title or description, with a back button to reset the search results.

Priority Colour Coding: Task priorities are visually highlighted using colours (green for low, amber for medium, and red for high) to make task importance easier to identify.

Delete Confirmation**:** A confirmation prompt appears before deleting a task to prevent accidental task removal.

Back Navigation: A back button is provided on the searching, creating and edit task pages, allowing users to easily return to the previous page without submitting changes.

Automated Unit Testing

Unit tests were implemented using pytest to verify the functionality of the application. Tests were created for user registration, login, task creation, editing, deletion, and retrieving tasks from the database.

Reflection and Learning Outcomes

This project provided practical experience in developing a web application using Python and the Flask framework. It demonstrated how backend logic, databases, and frontend templates work together to create a functional system. During the project, I gained a better understanding of Flask for building web applications, SQLAlchemy for database management, and pytest for automated testing. I also learned how Flask-Login can be used to manage user authentication and how Flask-WTF forms help handle form input and validation. Additionally, the project showed how Python can be used to create, retrieve, update, and delete data in a database using SQLAlchemy. Overall, the project helped strengthen my programming, testing, debugging, and web development skills.